GAMYAM

A
Data Structures Course Project Report

By

T.SNEHA 1602-24-737-176
P.SRAVAN 1602-24-737-178



Department of Information Technology
Vasavi College of Engineering (Autonomous)
ACCREDITED BY NAAC WITH 'A++' GRADE
(Affiliated to Osmania University)
Ibrahimbagh,
Hyderabad-500 031
2025

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University)

Hyderabad-500 031

Department of Information Technology



DECLARATION BY THE CANDIDATE

We, **T. Sneha** and **P. Sravan**, bearing hall ticket numbers **1602-24-737-176** and **1602-24-737-178**, hereby declare that the project report entitled **"GAMYAM"** is submitted in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering in Information Technology**.

This is a record of project work carried out by us and submitted in the form of a report

T. Sneha
Hall Ticket No: 1602-24-737-176
P. Sravan
Hall Ticket No: 1602-24-737-178

(Faculty In-Charge) **(Head, Dept of IT)**

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our Principal, Head of the Department, and all the faculty members of the Department of Information Technology for providing us with the necessary resources, guidance, and encouragement throughout the completion of our project titled "GAMYAM".

We extend our heartfelt thanks to our project guide for their continuous support, valuable suggestions, and constant motivation, which greatly contributed to the successful completion of this project.

We are also thankful to our friends and classmates for their cooperation and constructive feedback during various stages of this work.

Finally, we express our deep appreciation to our parents and family members for their constant support, inspiration, and encouragement.

T. Sneha
P. Sravan

Abstract

The rapid growth of real-world networks such as road maps, communication systems, and social networks has created a strong need for efficient graph-based computational tools. In many applications, users require solutions for identifying shortest paths, finding connectivity, detecting cycles, and visualizing the traversal of networks. Manual processing of such large and interconnected systems is difficult, time-consuming, and prone to errors. Therefore, there is a clear necessity for a software-based approach that can handle graph data, store it efficiently, and perform fast computations. The proposed project provides a modular solution that reads a graph from text files, constructs data structures such as adjacency lists and priority queues, and applies algorithms including Dijkstra's shortest path and BFS/DFS traversal. This design ensures efficient memory usage and fast execution even for large datasets. The system benefits users by providing clear insights into graph connectivity and optimal routing, which are essential in areas such as traffic navigation, network routing, and logistics planning. Researchers and developers can extend this framework by integrating more advanced algorithms such as A*, bidirectional search, or dynamic graph updates, making the project a solid foundational tool for academic studies and real-time applications.

.

INTRODUCTION

Graphs are among the most important data structures in computer science because they model relationships among objects. This project focuses on designing a graph-based system capable of storing node connectivity, computing shortest paths, detecting reachable components, and visualizing results through command-line output. The implementation uses C language and modular programming principles, enabling reusability, clarity, and ease of debugging.

PROBLEM STATEMENT

Many real-world systems require analysis of connections between entities. Without proper algorithms and data structures, operations like finding shortest routes, checking connectivity, or computing traversal sequences become inefficient. Therefore, the problem is to develop a program that:

- Reads and stores graph data efficiently
- Supports undirected/directed graphs
- Applies traversal and shortest-path algorithms
- Provides clear structured output to the user

DATA STRUCTURES USED

i) Adjacency List

The adjacency list is chosen to store graph data because it:

- Uses memory efficiently for sparse graphs
- Provides fast traversal of neighbors
- Allows easy modification of nodes and edges
- Supports BFS, DFS, and Dijkstra directly

ii)Priority Queue (Min-Heap)

Used for Dijkstra's algorithm because:

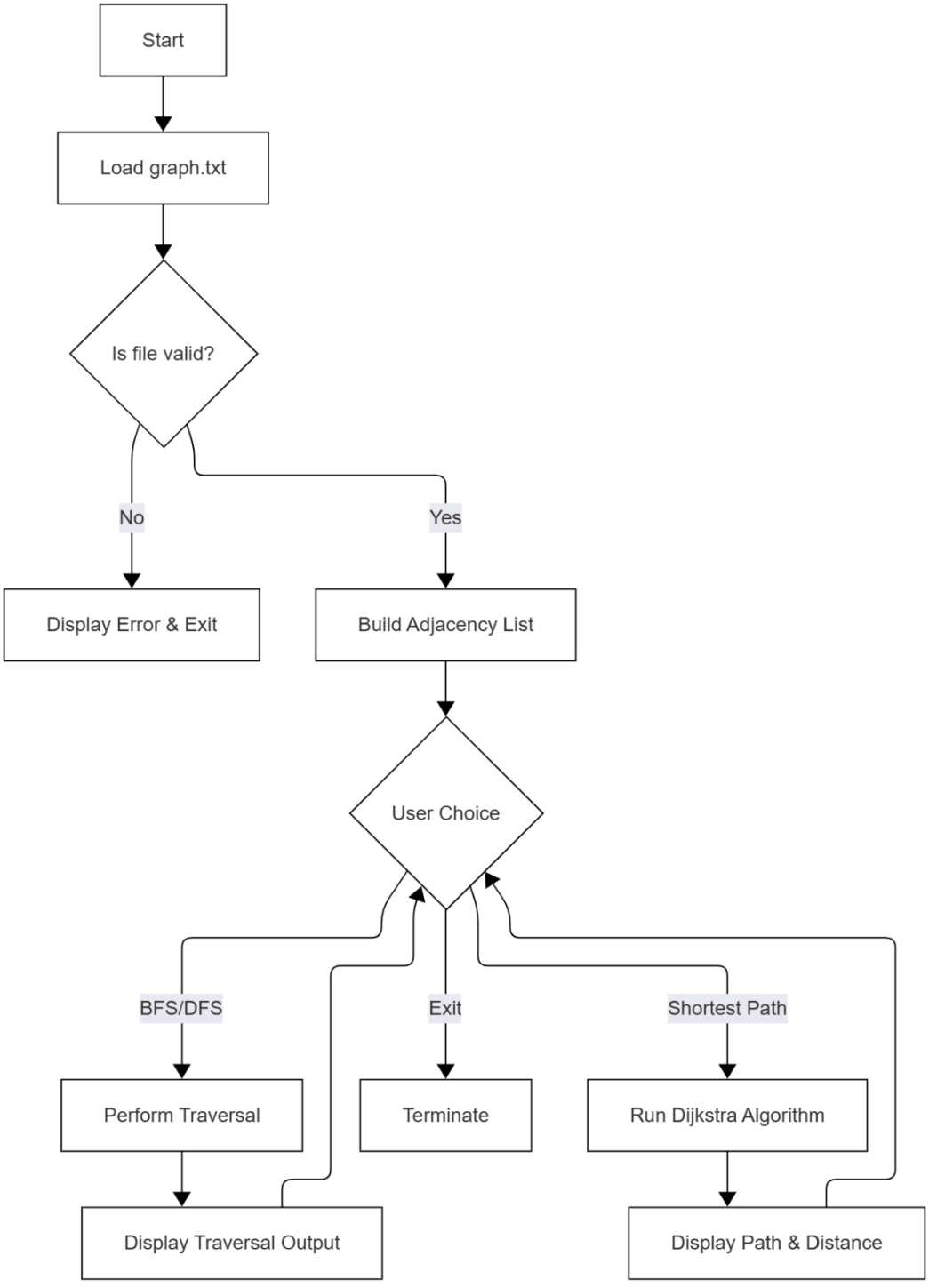
- It ensures extraction of minimum-distance node in O(log n)
- Greatly improves performance compared to linear search
- Allows scalable shortest-path computation

iii) Stack / Queue

Queue is required for BFS (level-order traversal)

- Stack is required for DFS (iterative version)
- Both support simple and efficient graph traversal

DESIGN – FLOWCHART



IMPLEMENTATION

Component	Minimum Requirement	Recommended
Processor (CPU)	Dual-core processor (Intel i3 / AMD equivalent)	Intel i5 or higher
RAM	2 GB	4 GB or higher
Storage	200 MB free space for project files	1 GB free for builds, logs, and extensions
Display	1024 × 768 resolution	1366 × 768 or higher
Input Devices	Keyboard, Mouse	–

Module 1 – Graph Storage & File Loading

This module:

- Reads graph.txt
- Parses nodes, neighbors, and weights
- Stores everything in an adjacency list

Key operations:

- Tokenizing node names
- Dynamic memory allocation
- Edge insertion
- Handling upper/lowercase names

Module 2 – Algorithms (BFS, DFS, Dijkstra)

This module contains:

BFS

- Uses queue
- Visits nodes level by level

DFS

- Uses stack
- Explores deeply first

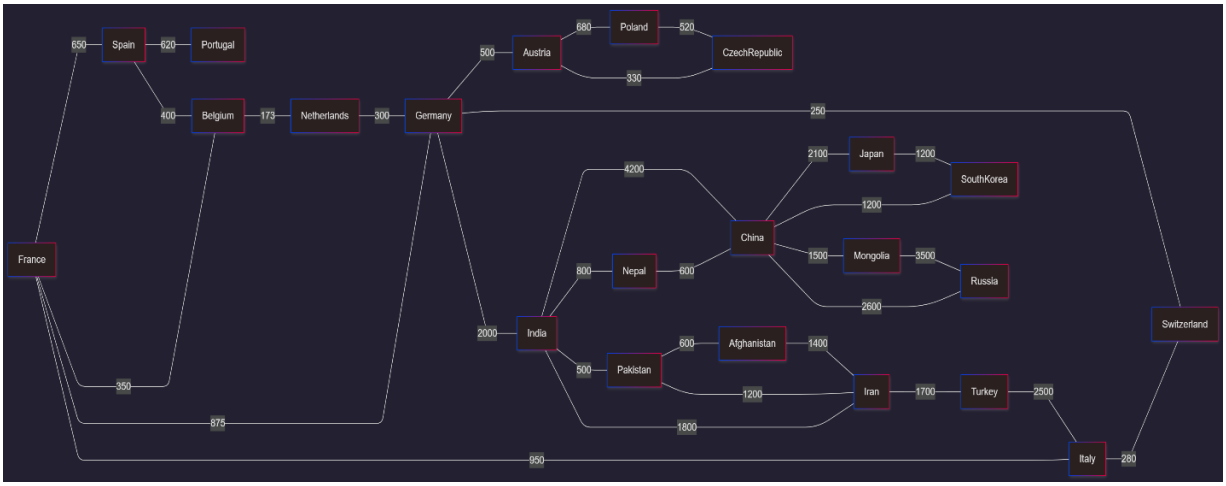
Dijkstra

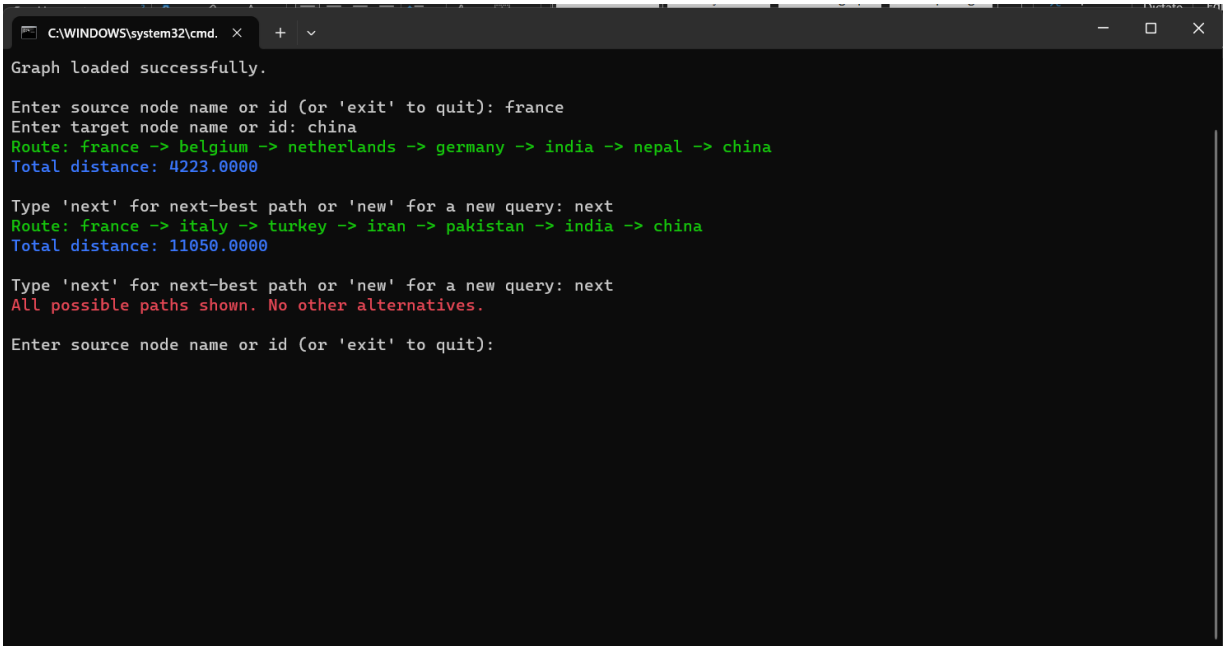
- Uses priority queue
- Computes shortest path based on weights

Error handling:

- No path exists
- Node not found
- Graph disconnected

Output


Spain
Portugal
Belgium
Netherlands
Germany
Austria
Poland
CzechRepublic
France
Switzerland
Italy
China
Japan
SouthKorea
Mongolia
Russia
Nepal
India
Pakistan
Afghanistan
Iran
Turkey
650
620
400
173
300
500
680
520
330
250
2100
1200
1200
1500
3500
2600
4200
800
600
2000
600
1400
500
1200
1700
2500
350
875
1800
950
280


C:\WINDOWS\system32\cmd.
Graph loaded successfully.
Enter source node name or id (or 'exit' to quit): france
Enter target node name or id: china
Route: france -> belgium -> netherlands -> germany -> india -> nepal -> china
Total distance: 4223.0000
Type 'next' for next-best path or 'new' for a new query: next
Route: france -> italy -> turkey -> iran -> pakistan -> india -> china
Total distance: 11050.0000
Type 'next' for next-best path or 'new' for a new query: next
All possible paths shown. No other alternatives.
Enter source node name or id (or 'exit' to quit):

Time Complexity Analysis

Algorithm	Time Complexity	Space Complexity
BFS	O(V + E)	O(V)
DFS	O(V + E)	O(V)
Dijkstra (Min-Heap)	O((V + E) log V)	O(V)
Adjacency List Build	O(V + E)	O(V + E)

Future Scope

- Add GUI visualization for graph
- Add support for A* algorithm
- Interactive path editing and dynamic graph updates
- Exporting graphs to JSON or CSV

Key Learnings

- Understanding of real-world graph modeling
- Efficient use of adjacency lists and heaps
- Modular programming in C
- Error handling in file I/O
- Algorithm optimization for large datasets

References

1. Thomas H. Cormen – *Introduction to Algorithms*
2. Mark Allen Weiss – *Data Structures & Algorithm Analysis in C*
3. GeeksforGeeks – Graph Algorithms
4. CLRS Dijkstra and BFS articles